

Python

Summary Session 5



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Что такое итерируемый объект?



Итерируемый объект

Это объект, который может состоять из множества элементов и предоставляет их по одному, когда это необходимо

Экспресс-опрос

?

Что такое цикл for?



Цикл for

Это конструкция, которая позволяет повторно выполнять блок кода (итерацию) для каждого элемента в последовательности.

Синтаксис



`for` переменная `in` последовательность: `# условие цикла`

`# блок кода, выполняющийся для каждого элемента`

Преимущества тернарного оператора:



Переменная — это переменная, которая на каждом шаге цикла присваивает значение текущего элемента из последовательности.



Последовательность — это итерируемый объект (например, строка), по которому цикл проходит.



Блок кода (или тело цикла) — это инструкции, которые будут выполняться для каждого элемента последовательности.

Цикл for



Условие цикла

Цикл for будет работать до тех пор, пока есть элементы в последовательности, которые можно обработать. Как только все элементы будут пройдены, цикл завершится.

Код

```
text = "Python"

for letter in text:

    print(letter)
```



Итерация

Это один шаг в процессе последовательного перебора элементов итерируемого объекта. На каждой итерации происходит обработка одного элемента из последовательности.

Экспресс-опрос

?

Что такое функция range?



Функция range

Это функция, которая используется для создания последовательности чисел, которые можно использовать в цикле. Она позволяет задавать диапазоны чисел и управлять шагом между ними.

Синтаксис range



```
range(start, stop, step)
```

Преимущества тернарного оператора



start (необязательный) — начальное значение (включительно). Если не указано, по умолчанию это 0.



stop (обязательный) — конечное значение (не включается в последовательность).



step (необязательный) — шаг, с которым создаётся последовательность. По умолчанию равен 1.

Функция range



(stop):

Если указать только конечное значение, последовательность начинается с 0 и идёт до указанного числа (не включительно).

Код

```
for i in range(5):  
    print(i)
```

Функция range



(start, stop):

Если указать два значения, последовательность начинается с первого (включительно) и идёт до второго числа (не включительно).

Код

```
for i in range(2, 6):  
    print(i)
```

Функция range



(start, stop, step):

Когда используется три аргумента, range() создаёт последовательность, начиная с числа start (включительно), заканчивая числом stop (не включается), с шагом step, который указывает, через сколько элементов нужно брать следующее число.

Код

```
for i in range(1, 10, 2):  
    print(i)
```

Функция range



range с отрицательным шагом:

Функция range() также поддерживает отрицательные значения для шага step, что позволяет создавать последовательности чисел в обратном порядке. В этом случае start должно быть больше stop, чтобы значения уменьшались с каждым шагом.

Отрицательный шаг:

```
for i in range(10, 0, -2):
    print(i)

for i in range(-4, -8, -1):
    print(i)
```

Экспресс-опрос

?

Что такое break, continue, else ?



break, continue, else

В Python цикле `for` можно использовать специальные операторы — `break`, `continue` и `else` — для управления выполнением цикла. Операторы работают также, как и в цикле `while`.

Полезно знать



Оператор break

Оператор break позволяет прервать выполнение цикла досрочно, как только будет выполнено определённое условие. Цикл завершится, даже если элементы в последовательности ещё остались.

Пример

```
for letter in "Python":
    if letter == "h":
        break # Останавливаем цикл, если найден
              символ "h"
    print(letter)
```

Полезно знать



Оператор continue

Оператор continue позволяет пропустить текущую итерацию цикла и перейти к следующей, не завершая сам цикл. Он используется, когда нужно игнорировать определённые элементы, но продолжить обработку остальных.

Пример

```
for letter in "Python":  
  
    if letter == "h":  
  
        continue # Пропускаем букву "h" и  
        продолжаем цикл  
  
    print(letter)
```


Полезно знать



Оператор else

Оператор else в цикле for выполняет блок кода, если цикл завершился нормально, без использования оператора break. Это полезно, когда нужно выполнить определённые действия, если цикл прошёл через все элементы без прерывания.

Пример

```
for letter in "Python":
    if letter == "a":
        break # Этот код никогда не выполнится,
так как "a" нет в строке
    print(letter)
else:
    print("Цикл завершён нормально.")
```

Пример с break и else:



```
for letter in "Python":  
    if letter == "h":  
        break # Цикл прерывается на символе "h"  
    print(letter)  
else:  
    print("Цикл завершён нормально.") # Этот блок не выполнится
```

Полезно знать



Вложенный цикл

```
for i in "AB":
    for j in "12":
        print(i, j)
```

С выводом времени

```
for hour in range(24):
    for minute in range(60):
        print("Время (часов:минут): ", hour,
              ': ', minute, sep='')
```

Экспресс-опрос



Что такое вложенные циклы?



Вложенные циклы

Это конструкции, в которых один цикл находится внутри другого. Вложенный цикл выполняется полностью для каждого прохода внешнего цикла.

Синтаксис вложенных циклов:



```
for внешняя_переменная in внешняя_последовательность:
    # код, выполняемый внутри внешнего цикла
    for внутренняя_переменная in внутренняя_последовательность:
        # код, выполняемый внутри обоих циклов
```



Вложенные циклы

Это не только комбинация двух `for` циклов, но также можно использовать комбинации `for` и `while` циклов для решения различных задач.

Вложенные циклы с использованием while и for



```
current_hours = int(input("Введите текущий час: ")) # Текущее время

hours = 3

end_time = current_hours + 3

while current_hours < 24 and current_hours < end_time: # Внешний цикл с использованием while
    for minutes in range(60): # Внутренний цикл с использованием for
        print("Время (часов:минут): ", current_hours, ':', minutes, sep='')

    current_hours += 1 # Увеличение значения часов на 1
```


Экспресс-опрос

?

Что такое коллекции?



Коллекции

Это структуры данных, которые позволяют хранить и организовывать множество элементов.



Изменяемые типы данных

Это такие структуры данных, которые позволяют изменять своё содержимое после создания объекта.



Список (list)

Это упорядоченная изменяемая (mutable) коллекция, которая может содержать элементы любых типов.

Основные характеристики списка



Изменяемость: Элементы списка можно изменять после его создания.



Упорядоченность: Элементы списка хранятся в том порядке, в котором они были добавлены.



Поддержка дубликатов: Список может содержать повторяющиеся элементы.



Поддержка различных типов данных: Список может содержать элементы разных типов.



Индексация: К элементам списка можно обращаться по индексу.

Пример создания списка



```
fruits = ["apple", "banana", "cherry"] # Список со строками
```

```
print(fruits)
```

```
numbers = [1, 2, 3] # Список с числами
```

```
print(numbers)
```

Хранение разных типов элементов в списке



Определение

Список (list) является универсальной структурой данных, которая может хранить элементы разных типов данных.

Код

```
mixed_list = [42, "Python", 3.14, True, [1, 2, 3]]  
  
print(mixed_list)
```

Создание списка с элементами



Определение

Список создаётся с использованием квадратных скобок [], а элементы внутри разделяются запятыми.

Код

```
my_list = [1, 2, 3, 4]  
  
print(my_list)
```


Создание пустого списка



Квадратные скобки

```
empty_list = []  
  
print(empty_list)
```

Функция list()

```
my_list = list()  
  
print(my_list)
```

Экспресс-опрос



Что такое индексация списков?



Индексация списков

Это процесс доступа к элементам списка по их позиции (индексу) внутри списка.

Пример индексации



```
my_list = ["apple", "banana", "cherry", "date"]
```

```
# Доступ к первому элементу
```

```
print(my_list[0])
```

```
# Доступ к последнему элементу с положительным индексом
```

```
print(my_list[3])
```

```
# Доступ к последнему элементу с отрицательным индексом
```

```
print(my_list[-1])
```

```
# Доступ к первому элементу с отрицательным индексом
```

```
print(my_list[-4])
```



Срезы списков (slices)

Это способ получения части списка (подсписка), без изменения исходного списка. Срезы позволяют выбрать элементы по индексу с указанием начала, конца и шага.

Основной синтаксис срезов



start (необязательно) — индекс начала среза (включительно). По умолчанию — 0.



stop (обязательно) — индекс конца среза (не включительно).



step (необязательно) — шаг, с которым нужно выбирать элементы. По умолчанию — 1.

Срезы положительным индексом



```
my_list = [0, 1, 2, 3, 4, 5, 6]
# Срез с 1-го по 4-й элемент (не включительно)
print(my_list[1:4])
# Срез каждого второго элемента
print(my_list[::2])
# Срез каждого второго элемента в обратном порядке
print(my_list[::-2])
# Срез от 4-го элемента до начала
print(my_list[4::-1])
# Срез с 3-го элемента до конца
print(my_list[3:])
# Копия списка с помощью среза
print(my_list[:])
```

Срезы отрицательным индексом



```
my_list = [0, 1, 2, 3, 4, 5, 6]

# Срез от -4-го до -1-го элемента
print(my_list[-4:-1])

# Срез от -2-го до -5-го элемента
print(my_list[-2:-5:-1])

# Срез списка в обратном порядке
print(my_list[::-1])
```


Экспресс-опрос

?

Что такое изменение по индексу?



Изменение по индексу

В Python элементы списка можно изменять напрямую, используя индексацию.

Пример изменения элементов



```
my_list = ["apple", "banana", "cherry"]
```

```
# Изменим второй элемент (с индексом 1)
```

```
my_list[1] = "blueberry"
```

```
print(my_list)
```

Изменение по срезу



```
my_list = [10, 20, 30, 40, 50]
```

```
# Заменяем второй и третий элементы
```

```
my_list[1:3] = [200, 300]
```

```
print(my_list)
```

Использование отрицательных индексов для изменения



```
my_list = [100, 200, 300, 400]
```

```
# Заменяем последний элемент
```

```
my_list[-1] = 500
```

```
print(my_list)
```

Экспресс-опрос

?

Чему служат операции со списками?



Операции со списками

Списки в Python поддерживают множество операций, позволяющих изменять, объединять или проверять списки. Ниже приведены основные операции со списками.

Конкатенация списков (объединение)



```
list1 = [1, 2, 3]

list2 = [4, 5, 6]

combined_list = list1 + list2

print(combined_list)
```


Повторение списка



```
repeated_list = [0] * 5  
  
print(repeated_list)  
  
my_list = [1, 2, 3]  
  
repeated_list = my_list * 3  
  
print(repeated_list)
```

Проверка на наличие элемента в списке



```
my_list = [1, 2, 3, 4, 5]
```

```
print(3 in my_list)
```

```
print(6 in my_list)
```

```
my_list = ["apple", "banana", "cherry"]
```

```
print("apple" in my_list)
```

```
print("app" in my_list) # Ищет полное совпадение элемента
```

Длина списка



```
my_list = [1, 2, 3, 4, 5]
```

```
print(len(my_list))
```

Полезно знать



Из строки

```
word = "python"

my_list = list(word # Строка "разбивается" на
отдельные символы

print(my_list)

print(type(my_list))
```

Из объекта range:

```
my_range = range(1, 11, 2)

print(my_range)

print(type(my_range))

my_list = list(my_range)

print(my_list)
```



Сравнение списков

Списки можно сравнивать друг с другом с помощью операторов сравнения.

Примеры



```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
list3 = [1, 3, 0]

# Сравнение на равенство
print(list1 == list2)
print(list1 == list3)

# Сравнение на неравенство
print(list1 != list2)
print(list1 != list3)

### Сравнение на больше/меньше, больше или равно/меньше или равно
print(list1 < list2)
print(list3 > list1)
print(list1 <= list2)
print(list1 >= list3)
```



Поддержка операции

Для корректного сравнения списков, элементы должны быть одного типа или поддерживать операцию сравнения между собой.

Примеры



```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
list3 = [1, 3, 0]

# Сравнение на равенство
print(list1 == list2)
print(list1 == list3)

# Сравнение на неравенство
print(list1 != list2)
print(list1 != list3)

### Сравнение на больше/меньше, больше или равно/меньше или равно
print(list1 < list2)
print(list3 > list1)
print(list1 <= list2)
print(list1 >= list3)
```


Пример корректного сравнения



```
list1 = [1, 2.6, 2]
```

```
list2 = [1, 2, 3]
```

```
# int и float можно сравнивать между собой
```

```
print(list1 < list2)
```

Пример ошибки типов



```
list1 = [1, 2, "apple"]
```

```
list2 = [1, 2, 3]
```

```
# Попытка сравнить числовое значение с строкой вызовет ошибку TypeError
```

```
print(list1 < list2)
```

Пример ошибки типов



```
Traceback (most recent call last): @ Explain with AI
  File "/home/tanya/PycharmProjects/pythonProgramItch/\_notes/test.py", line 5, in <module>
    print(list1 < list2)
          ^^^^^^^^^^^^^
TypeError: '<' not supported between instances of 'str' and 'int'

Process finished with exit code 1
```



Особенности сравнения

Важно учитывать, что при первом нахождении отличающихся элементов интерпретатор остановит сравнение. Поэтому если после них будут находиться несравнимые типы это не приведет к ошибке.

Пример



```
list1 = [1, 2, "apple"]
```

```
list2 = [1, 3, 10]
```

```
print(list1 < list2) # Вернёт True, так как 2 < 3, сравнение остановится до строки "apple"
```

Экспресс-опрос

?

Чему служит цикл for со списками?

Пример



```
my_list = [1, 2, 3, 4, 5]
```

```
for item in my_list:
```

```
    print(item)
```



Цикл for со списками

Вложенные циклы for позволяют перебирать элементы списка, где каждый элемент сам по себе является итерируемым объектом, например, строкой. Внутренний цикл проходит по символам каждой строки.

Вложенный цикл for по списку строк



```
my_strings = ["apple", "banana", "cherry"]

for word in my_strings:
    for letter in word:
        print("letter:", letter)

print()
```



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Таблица умножения

Напишите программу, которая выводит таблицу умножения для чисел от 1 до n. Где n это число, которое введет пользователь. Оформите вывод так, чтобы столбцы были равные (число ровно под числом)

Пример вывода:

Введите число: 5

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

Домашнее задание

2. Лестница чисел

Напишите программу, которая принимает число n и выводит "лестницу" из чисел. Каждая строка лестницы содержит числа от 1 до номера строки.

Пример вывода:

Введите число: 5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Домашнее задание

3. Удаление всех повторяющихся символов

Напишите программу, которая принимает строку и удаляет из неё все повторяющиеся символы, сохраняя только первые их вхождения.

Пример вывода:

Введите строку: Python programming

Результат: Python prgami



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Создание списка с пользовательскими данными

Напишите программу, которая принимает список целых чисел от пользователя и добавляет каждое число в динамический массив. Программа должна принимать новое число от пользователя и добавлять его в динамический массив до тех пор, пока пользователь не введет команду "Выход". Выведите итоговый динамический массив на экран с помощью команды print.

Пример вывода:

```
Введите число или 'выход' :: 1
Введите число или 'выход' :: 2
Введите число или 'выход' :: выход
Итоговый динамический массив: [1, 2]
```


Домашнее задание

2. Квадрат нечетных чисел

Напишите программу, которая изменяет изначальный список и возводит в квадрат только нечётные числа.

```
numbers = [4, 9, 1, 7, 2, 5, 0, 3, 7, 1, 3]
```

Пример вывода:

Изначальный список: [4, 9, 1, 7, 2, 5, 0, 3, 7, 1, 3]

Измененный список: [4, 81, 1, 49, 2, 25, 0, 9, 49, 1, 9]

Заключение

